

8.8小组模拟面

1.为什么使用rtmp，而不其他的流媒体协议？

rtmp（实时流媒体传输协议）具有广泛的兼容性，很多流媒体服务器都为rtmp提供了很好的支持，且rtmp的配置相对简单，既可以用作点播也能用于直播，其他的流媒体协议比如rtsp（实时视频流传输），一般是用作摄像头，监控，或专业拍摄的数据实现直播，底层采用UDP，允许大量丢包。之所以我选择RTMP，因为它采用TCP作为底层，我不希望网络因素造成问题希望去排查，所以写了一个比较简单的版本。同时在后续实现功能过程中，我也发现有一些更适合视频点播的协议，比如hls协议通过拆分视频成ts小块，通过m3u8索引找到具体播放位置，方便跳转，**且可以通过转码实现自适应码率，实现客户端网络状态和播放源的匹配**。如果要做直播的话，我还可以用rtmp进行推流，http-flv进行拉流，或者用webrtc用于直播模块，但是nginx上没有相关配置，需要用到SRS，所以我没单独考虑

总结来说：

rtmp最好用在直播推流

http-flv：用于直播拉流

hls用于点播

webrtc / rtsp用于视频会议，直播：前者需要配置SRS流媒体服务器，后者一般是摄像头拍摄的数据进行直播

协议	传输层	延迟	典型场景	优势	劣势
RTMP	TCP	1~3秒	直播推流（OBS等）	简单易用，广泛支持	延迟较高，浏览器不支持
RTSP	UDP/TCP	<1秒	摄像头监控、视频会议	实时性强	兼容性差，依赖专用客户端
HLS	HTTP/TCP	5~20秒	点播、直播回放	自适应码率，兼容所有设备	延迟高
WebRTC	UDP	<1秒	实时互动（连麦、通话）	超低延迟，抗网络波动	实现复杂
SRT	UDP	1~3秒	专业直播（抗丢包）	高抗丢包能力，适合弱网环境	生态较新

2.什么是流式传输，和普通下载的区别

流式传输就是数据在传输时就能被播放，而不需要等待整个文件下载完成，普通下载在把整个视频下载后才能进行播放，相比于普通下载，流式传输的优势在于延迟更低，不需要更大的存储空间（不用等待全部下载完），支持边下边播。一般的音视频

3.流媒体服务器会缓存所有数据吗？

不会，流媒体服务器一般是用作直播实时数据流的，双端分别推流和拉流，点播的本质也是打开视频，把视频变成数据流供拉流方获取，比如rtmp协议，播放视频的时候就是主线程一直在拉取，然后放入队列，跳转的时候如果是向后跳转，在项目中采取的逻辑是拉流并丢弃不是目标时间戳的数据，实际上这种方式假如网络比较差，缓冲数据不够时，会强制下载完所有的中间数据（1秒跳到10秒就拉取9秒的，假如数据只到了4秒还得等后面的数据），正确处理方式应该是像往前跳那样，调整拉流的起始位置时间戳，丢弃队列中全部包，然后开始继续拉流

除了rtmp是实时的视频流，假如直播时主播端把数据按rtmp推流到了nginx服务器，此时也不会缓存数据，假设用http-flv格式推流，此时nginx只会做一下转码的操作，把数据重新封装并等待拉取，也就是客户端用什么流拉取，流媒体服务器才进行转码成对应数据。只有hls是会分片变成静态资源的缓存在流媒体服务器（一般是定义若干个小块，比如6个，然后每个2秒，进行循环覆盖（直播的场景））

在直播中，流媒体服务器会保存最近的I帧，供出现延迟的客户端拉取

4.直播发生延迟，怎么去改善（弱网环境下的直播优化）

由于直播的全过程是从主播端数据的封装，再推流通过网络传输到流媒体服务器，流媒体服务器转码，最后客户端拉流，所以可以从每个环节分别去考虑

- 从主播端，可以采用压缩率更高的算法，比如H265来代替H264，更高的压缩比，方便后续网络传输。同时弱网环境下还能适当丢弃P帧，减少网络带宽压力
- 在传输层面考虑，可以采用更好的传输协议或者流媒体协议，比如webRTC和QUIC
- 在客户端层面，可以配合服务器，实现CDN加速，拉取最近的服务器上的资源，也可以动态调整码率分辨率（可以用hls自适应码率），还可以增设缓存队列，应对网络抖动
- 削弱累计延迟带来的影响：假如客户端切到后台，此时直播的位置和最新位置不一致，一般有两种方式解决，一是客户端切回来时重新建立连接和拉流，二是在后台时只接收音频数据，回到前台时视频与音频看齐，直接丢弃，直到同步。

5.缓冲雪崩，击穿和穿透

缓存雪崩

定义：为了保证缓存数据和数据库数据一致，会给redis数据设置过期时间，当缓存数据过期时如果用户访问缓存数据，系统会访问数据库并把数据同步到缓存。**所谓缓存雪崩就是大量缓存在同一时间过期或失效，或者redis宕机时有大量用户请求，导致无法在redis中处理，大量请求直接访问数据库，导致数据库宕机，进一步导致系统崩溃**

如何解决：

- 均匀设置过期时间，避免大量数据短时间同时过期
- 互斥锁：如果访问的数据不在redis，就加个互斥锁，保证同一时间**只有一个请求**来构建缓存（实现互斥锁时需要设置超时时间，避免请求发生时上锁后阻塞，出现死锁）
- 后台更新缓存：让缓存数据永久有效，更新缓存工作交给后台线程定期实现（即便设置了永久有效也不意味数据一直在内存，当系统内存紧张时会淘汰部分缓存数据，而淘汰工作和更新工作进行时，业务线程读取缓存失败就会返回空值，从而被视为数据丢失）

对于通知后台线程，主要有两种方法：

1.后台线程定期检测缓存是否有效，失效后从数据库中读数据并更新到缓存（更新时间有间隔，需要后台线程检测然后更新，用户体验受影响），这是后台线程定时检测法

2.业务线程发现缓存数据失效后，通过消息队列发送消息通知后台线程更新缓存（在业务上线前就可以提前缓存（缓存预热），所以用户体验比较好），这是业务线程检测，并通知后台线程更新的方法

- 故障宕机引起缓存雪崩解决方法

1.服务熔断或请求限流：暂停业务应用对缓存的访问，同时启用限流机制，只把少部分请求交给数据库处理

2.构建redis缓存高可靠集群：通过主从节点方式构建redis缓存高可靠集群

缓存击穿

定义：即热点数据频繁被访问，如果热点数据过期，则大量请求会冲垮服务器

如何解决：

- 互斥锁方案：保证同一时间只有一个业务线程更新缓存（同雪崩）
- 不给热点数据设置过期时间，由后台异步更新缓存，或者在过期前提前通知后台线程重新更新缓存并设置过期时间

缓存穿透

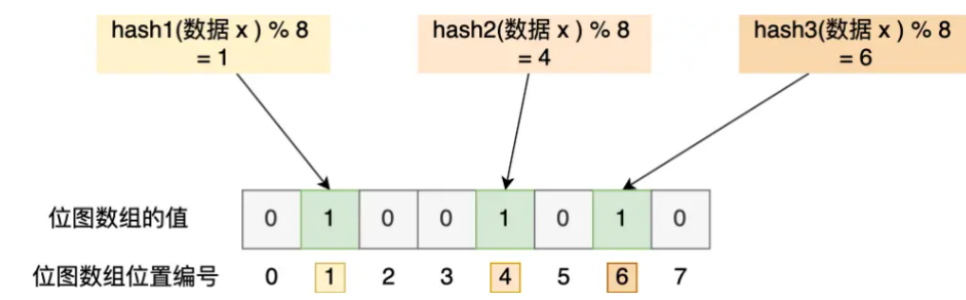
定义：访问的数据既不在缓存也不在数据库中，大量请求到来时数据库压力剧增，导致缓存穿透

缓存穿透发生场景：业务误操作（误删除），黑客恶意攻击

如何解决：

- 非法请求限制
- 缓存空值（在发现缓存穿透时，将请求数据设置为空）
- 布隆过滤器

初始值为0的位图数组和N个哈希函数，当我们往数据库写入数据时，通过哈希函数计算得到N个哈希值，并把位图数组对应位置置为1



如果对应位为1，则数据可能存在，有一位不得1一定不存在（快速过滤掉一批数据）

总结：

缓存异常	产生原因	应对方案
缓存雪崩	大量数据同时过期	- 均匀设置过期时间，避免同一时间过期 - 互斥锁，保证同一时间只有一个应用在构建缓存 - 双 key 策略，主 key 设置过期时间，备 key 永久，主 key 过期时，返回备 key 的内容 - 后台更新缓存，定时更新、消息队列通知更新
	Redis 故障宕机	- 服务熔断 - 请求限流 - 构建 Redis 缓存高可靠集群
缓存击穿	频繁访问的热点数据过期	- 互斥锁 - 不给热点数据设置过期时间，由后台更新缓存
缓存穿透	访问的数据既不在缓存，也不在数据库	- 非法请求的限制； - 缓存空值或者默认值； - 使用布隆过滤器快速判断数据是否存在；

6.左前缀原则（组合索引中最左匹配原则）

即要用到组合索引，必须保证where判断条件中，靠左的必须连续用到，比如（a，b，c），只用b不走索引，用了a才会走，假如用了ab，在开启优化器前提下，先用哪个效率都一样（（a，b）和（b，a）），否则一定是顺序使用才能用到索引

7.线程池拒绝策略

触发时机？

当线程池的任务缓存队列已满并且线程池中的线程数目达到最大线程数时，如果还有任务到来就会采取任务拒绝策略，

四种线程池拒绝策略

java中内置了以下四种策略：

ThreadPoolExecutor.AbortPolicy:丢弃任务并抛异常
RejectedExecutionException。

ThreadPoolExecutor.DiscardPolicy：丢弃任务，但是不抛出异常。

ThreadPoolExecutor.DiscardOldestPolicy：丢弃队列最前面的任务，然后重新提交被拒绝的任务

ThreadPoolExecutor.CallerRunsPolicy： **由提交任务的线程处理该任务**

C++并没有内置线程池，所以也没有内置拒绝策略，但是这四个策略已经涵盖大部分情况，因此自定义线程池拒绝策略的实现可以模仿这四个策略